

Procedure Calls

Review

- **3 formats of MIPS instructions** in binary:
 - Op field determines format

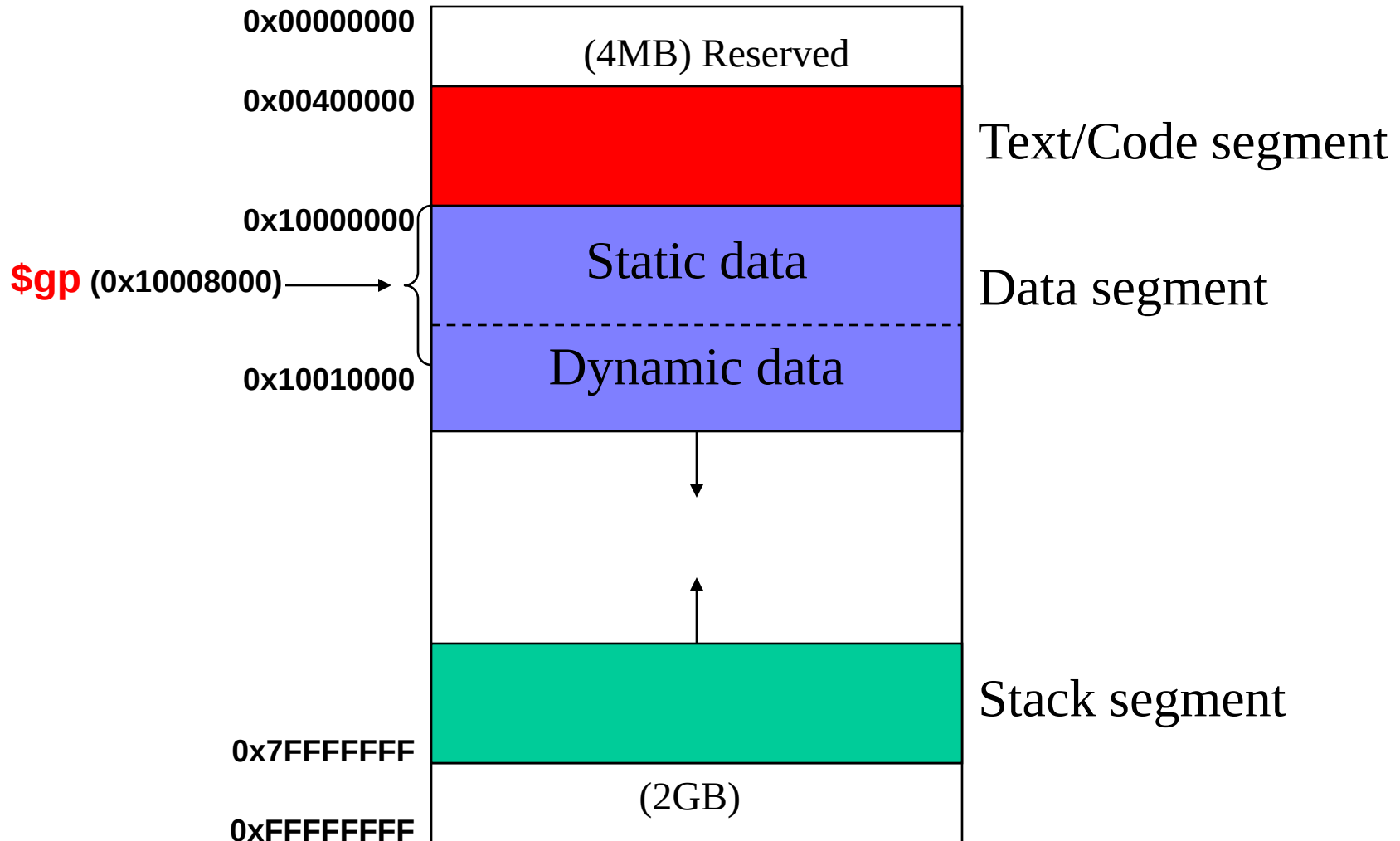
	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
R	op	rs	rt	rd	shamt	funct
I	op	rs	rt	immediate		
J	op	destination address				

- **Operands**
 - **Registers:** \$0 to \$31 mapped onto: \$zero, \$at, \$v_, \$a_, \$s_, \$t_, \$gp, \$sp, \$fp, \$ra
 - **Memory :** Mem[0], Mem[4], ... , Mem[4294967292]
 - Index is the “address” (Array index => Memory index)
- **Stored program concept** (instructions are numbers!)

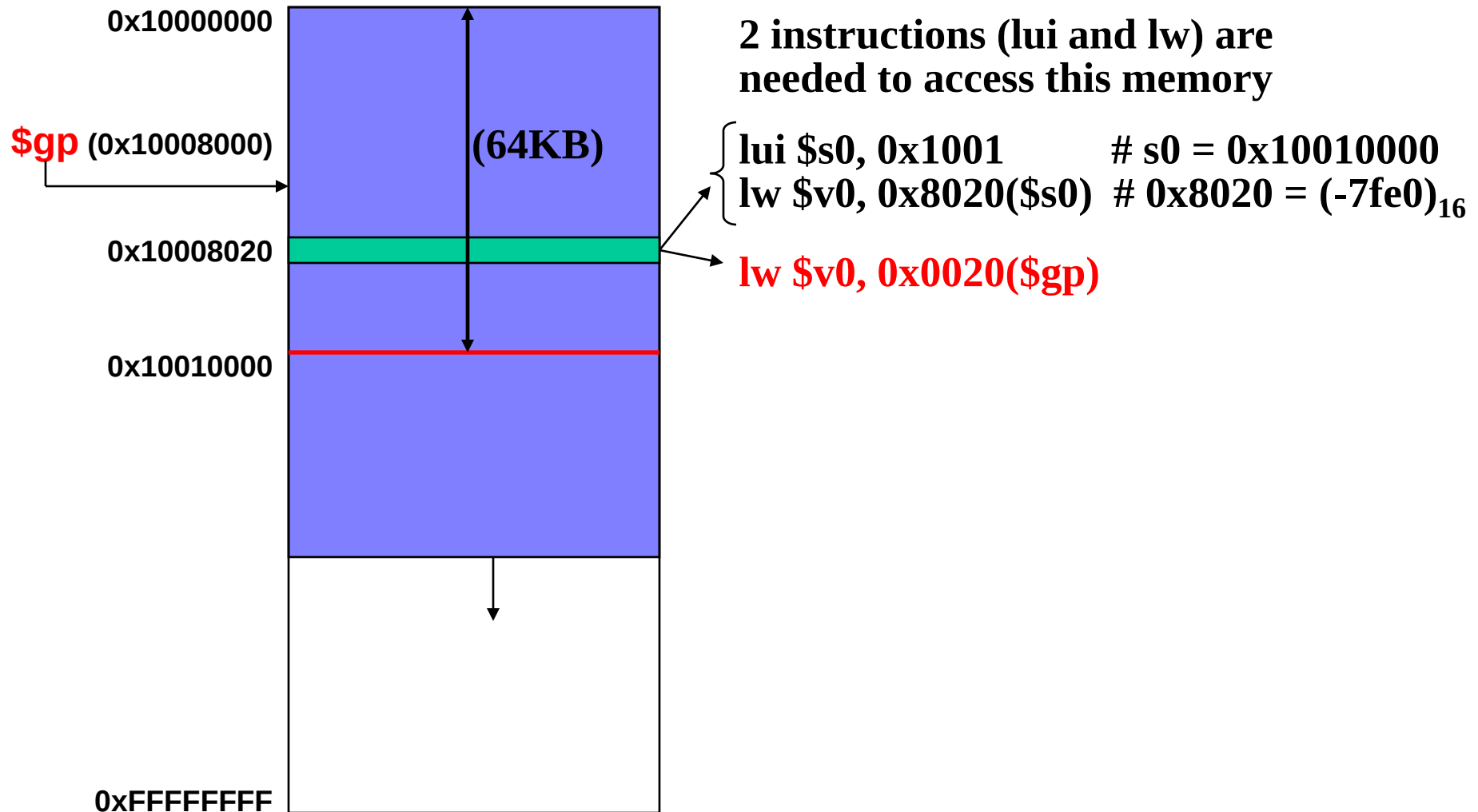
Overview

- Memory layout
- Stack
- C functions/procedures
- MIPS support (instructions) for procedure calls

Memory Layout



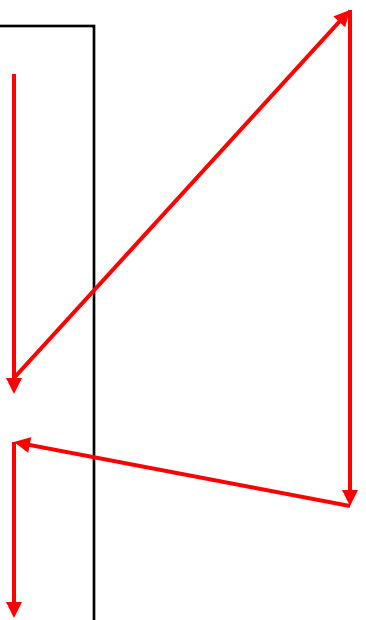
Data Segment



C Functions / Procedures

```
main() {  
    int i, j, k;  
    ...  
    i = fact(j,k);  
    ...  
}
```

```
int fact (int mcand, int mlier) {  
    int product;  
  
    product = 0;  
    while (mlier > 0) {  
        product = product + mcand;  
        mlier = mlier -1; }  
    return product;  
}
```



The diagram illustrates the execution flow between the `main()` function and the `fact` function. A red arrow points from the `i = fact(j,k);` line in the `main()` block to the opening curly brace of the `fact` function. Another red arrow points from the closing curly brace of the `fact` function back to the line immediately following `i = fact(j,k);` in the `main()` block, representing the return of control and data.

What information must compiler keep track of?

Procedure Call Bookkeeping

- **Problems**

- Procedure address
- Return address
- Arguments
- Return value
- Local variables

- **Register conventions**

Labels

\$ra

\$a0, \$a1, \$a2, \$a3

\$v0, \$v1

\$s0, \$s1, ..., \$s7

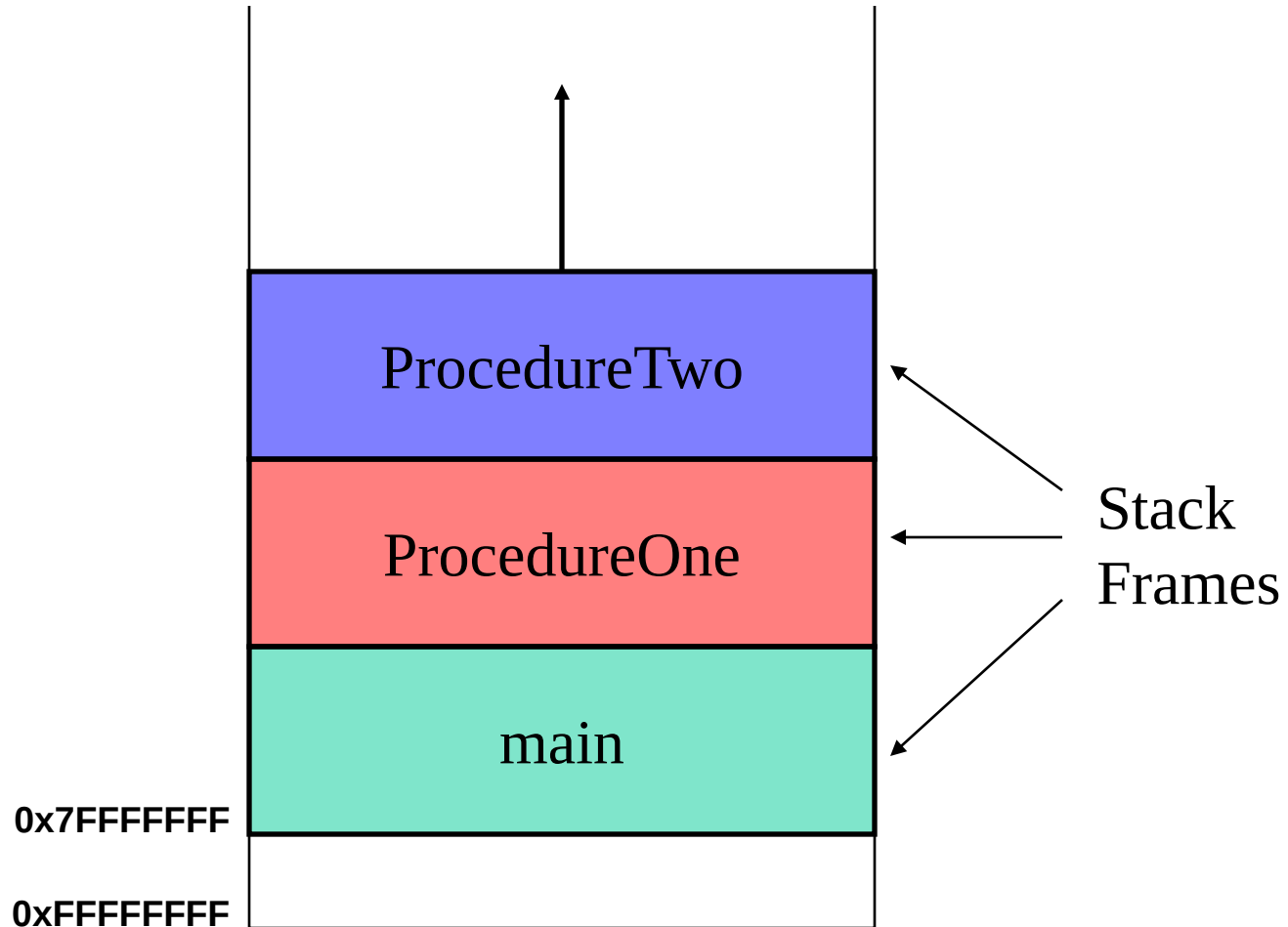
- **Dynamic nature of procedures**

- Procedure call **frames**
 - Arguments, save registers, local variables

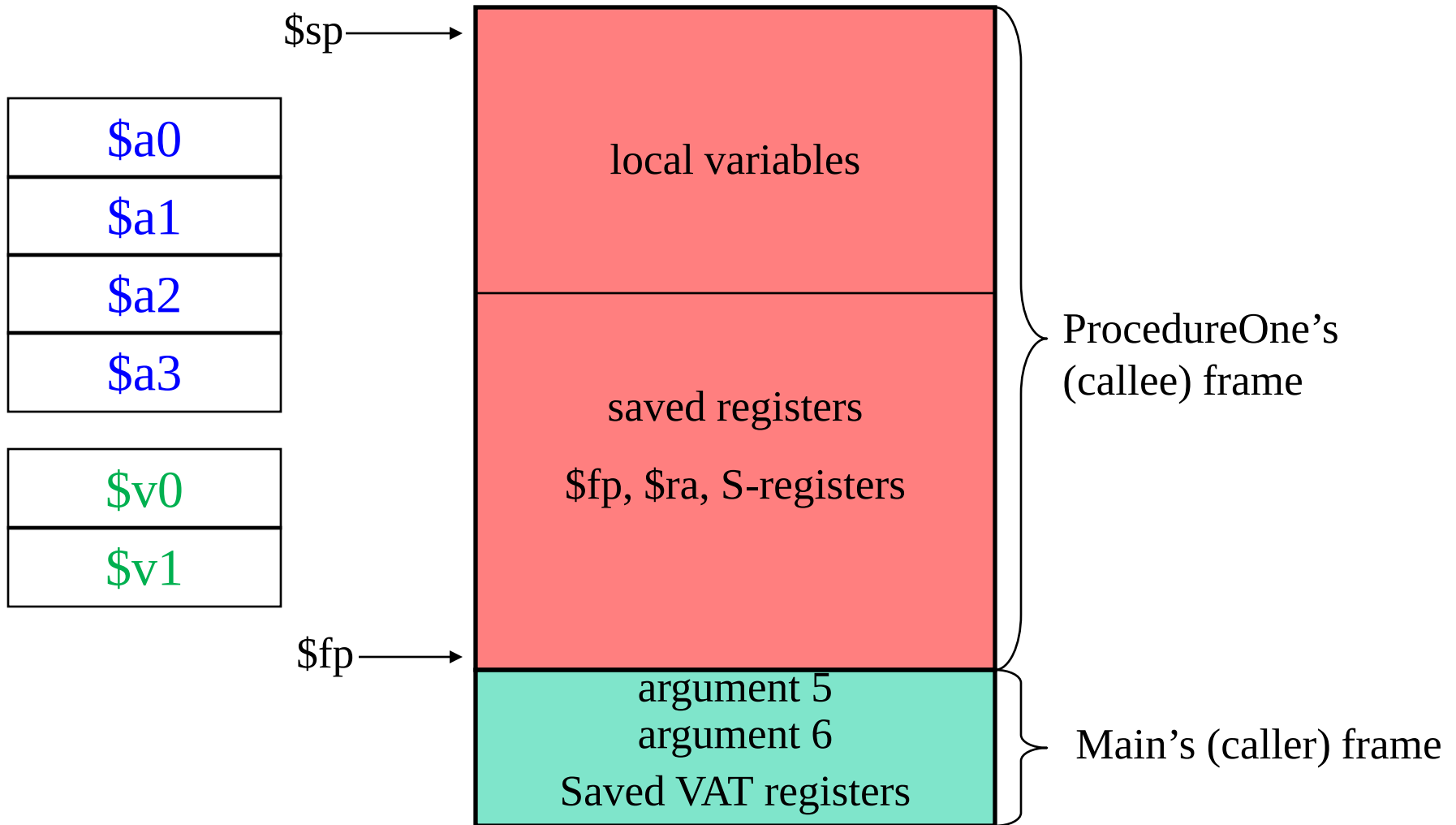
Registers Preserved on Calls

Name	Register Number	Usage	Preserved on call
\$zero	0	the constant value 0	n.a.
\$at	1	reserved for the assembler	n.a.
\$v0-\$v1	2-3	expr. evaluation and function result	no
\$a0-\$a3	4-7	arguments (procedures/functions)	yes
\$t0-\$t7	8-15	temporaries	no
\$s0-\$s7	16-23	saved	yes
\$t8-\$t9	24-25	more temporaries	no
\$k0-\$k1	26-27	reserved for the operating system	n.a.
\$gp	28	global pointer	yes
\$sp	29	stack pointer	yes
\$fp	30	frame pointer	yes
\$ra	31	return address	yes

The Stack



Stack Frames



Caller / Callee Conventions

- **Immediately before the caller invokes the callee**
 - Pass arguments (\$a0 - \$a3). Extra args: push on the stack
 - Save caller-saved registers (\$a0 - \$a3; \$t0 - \$t9)
 - Execute **jal** (jumps and links to callee; saves return addr)
- **Just before the callee starts executing**
 - Allocate memory for the frame ($\$sp = \$sp - \text{fsize}$)
 - Save callee-saved registers (\$s0-\$s7; \$fp; \$ra)
 - $\$fp = \$sp + (\text{fsize} - 4)$
- **Immediately before the callee returns to caller**
 - Place the returned value in register \$v0
 - Restore all callee-saved registers
 - Pop the stack frame ($\$sp = \$sp + \text{fsize}$); restore \$fp
 - Return by jumping to the address in \$ra

Procedure Support

```
main( ) {  
    ...  
    s = sum (a, b);  
    ...  
}
```

```
int sum(int x, int y) {  
    return x + y;  
}
```

address

```
1000 add    $a0, $s0, $zero    # $a0 = x  
1004 add    $a1, $s1, $zero    # $a1 = y  
1008 addi   $ra, $zero, 1016    # $ra=1016  
1012 j      sum                # jump to sum  
1016 ...  
  
2000 sum:   add    $v0, $a0, $a1  
2004 jr    $ra                # jump to 1016
```

Jump and Link Instruction

- Single instruction to jump and save return address
 - **jal**: jump and link (*Make the common case fast*)
 - J Format Instruction: **jal label**
 - Should be called *laj*
 1. (link): save address of next instruction into \$ra
 2. (jump): jump to label

```
1000 add    $a0,$s0,$zero    # $a0 = x
1004 add    $a1,$s1,$zero    # $a1 = y
1008 jal    sum    # $ra = 1012; jump to sum
1012 ...
2000 sum:   add    $v0,$a0,$a1
2004 jr    $ra                # jump to 1012
```

Nested Procedures

```
int sumSquare(int x, int y) {  
    return mult(x,x) + y;  
}
```

sumSquare:

```
    subi $sp,$sp,12
```

```
    sw $ra, 8($sp)
```

```
    sw $a0, 0($sp)
```

```
    sw $a1, 4($sp)
```

```
    addi $a1,$a0,$zero
```

```
    jal mult
```

```
    lw $ra, 8($sp)
```

```
    lw $a0, 0($sp)
```

```
    lw $a1, 4($sp)
```

```
    add $v0,$v0,$a1
```

```
    addi $sp,$sp,12
```

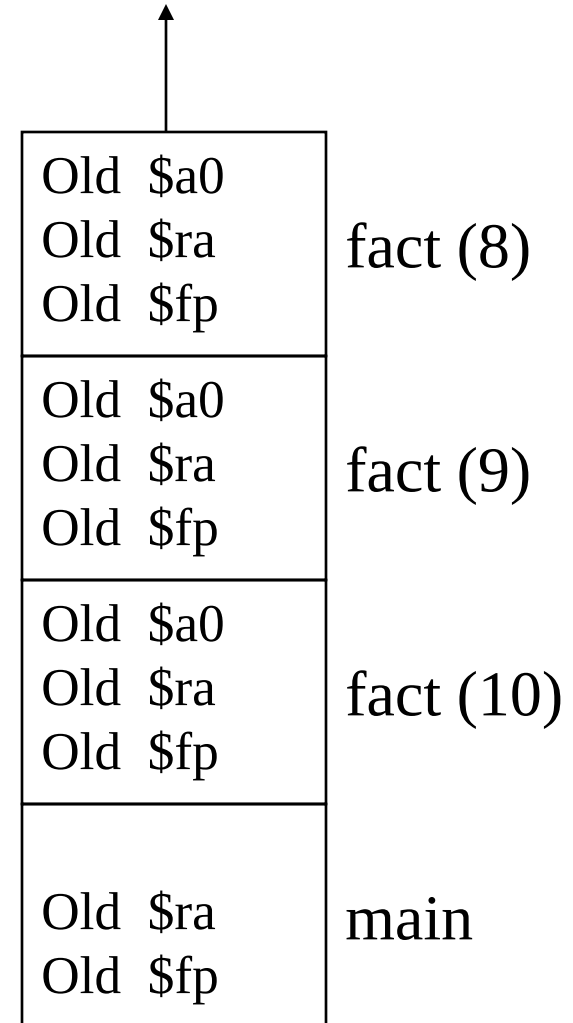
```
    jr $ra
```

```
# Begin MIPS Code  
# allocate stack frm  
# PUSH: save rtn addr  
# PUSH: save x  
# PUSH: save y  
# mult(x,x)  
# call mult w/ jal  
# POP: get rtn addr  
# POP: restore x  
# POP: restore y  
# mult()+y  
# free stack frame  
# return to caller
```

Example (1/2)

```
main( ) {  
    int f;  
    f = fact (10);  
    printf (“Fact(10) = %d\n”, f);  
}
```

```
int fact ( int n) {  
    if (n < 1)  
        return (1);  
    else  
        return (n * fact(n-1));  
}
```



Example (2/2)

```
main:  subu    $sp, $sp, 32
       sw     $ra, 20($sp)
       sw     $fp, 16($sp)
       addiu  $fp, $sp, 28
       li     $v0, 4
       la     $a0, str
       syscall
       li     $a0, 10
       jal    fact
       addu   $a0, $v0, $zero
       li     $v0, 1
       syscall
       lw     $ra, 20($sp)
       lw     $fp, 16($sp)
       addiu  $sp, $sp, 32
       jr     $ra
```

```
fact:  subu    $sp, $sp, 32
       sw     $ra, 20($sp)
       sw     $fp, 16($sp)
       addiu  $fp, $sp, 28
       sw     $a0, 0($fp)
       lw     $v0, 0($fp)
       bgtz   $v0, L2
       li     $v0, 1
       j      L1
L2:    lw     $v1, 0($fp)
       subu   $v0, $v1, 1
       move   $a0, $v0
       jal    fact
       lw     $v1, 0($fp)
       mul    $v0, $v0, $v1
L1:    lw     $ra, 20($sp)
       lw     $fp, 16($sp)
       addiu  $sp, $sp, 32
       jr     $ra
```


Summary

- Caller / callee conventions
 - Rights and responsibilities
 - Callee uses VAT registers freely
 - Caller uses S registers without fear of being overwritten
- Instruction support: `jal label` and `jr $ra`
- Use stack to save anything you need. *Remember to leave it the way you found it*
- Register conventions
 - Purpose and limits of the usage
 - Follow the rules even if you are writing all the code yourself